

完全模型组八邻域算法，拐弯，十字，以及 AI 元素处理

奶龙说的队 温麟陇

为了便于我撰写课程报告，这个其实写的不是很细致，可以参考一下思路。当然，所有室内赛道几乎都可以参考这个思路，因为都是蓝白配赛道。每年完全模型组都是基础赛道+AI 元素。基础赛道中主要是要处理拐弯，圆环，十字。圆环较难，我把十字的完整思路都展示出来了，便于大家参考，十字的很多思路可以用在圆环（找点和特征），这也便于大家去完成十字和圆环的处理。另外，我大部分都是把代码删掉了，只留下了理论，之后我们会给大家一部分代码，足够让小车动起来，这部分理论讲解如果大家吸收了，并且理解了我们给的部分代码，花时间是完全可以完成其余部分代码的。另外，这份资料也可以帮助大家了解一下智能车竞赛的部分赛道，在板块 1 中，给出了整体方案，这也是大家需要学习的内容，注意：完全模型组与其他赛道不同，主要是上位机 C++ 开发，嵌入式弱化一些，如果想要承担软件设计的同学这份资料会有一定帮助。

请勿把这份资料外传!!!

1. 研究总体方案

智能车采用传统的机器人系统架构，以上位机作为控制核心，传递指令给到下位机，借由下位机控制传感器、电机和舵机等，进而控制车辆的机械结构使得车辆完成运动。

智能车上位机采用百度官方提供的 Edgeboard 算力板，下位机采用英飞凌提供的 TC264 单片机，整个车身采用赛曙科技提供的 I 型车模。整个智能车从架构上看符合机器人系统的设计，与本次实践内容高度契合，从车模本身来看，简化的同时并未丧失真实车辆应当具备的结构。整车采用后驱设计，由舵机带动前总成转向，电机带动差速器进而带动后轮转动。整车架构示意如图 2.1 所示。

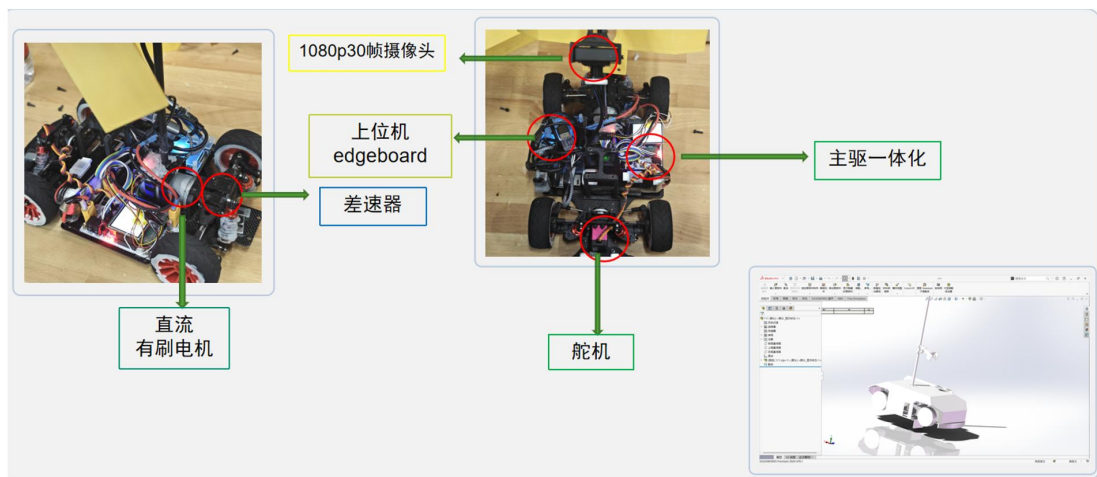


图 2.1 整车架构示意图

智能车在真实环境中，利用摄像头感知到外界环境，传递给上位机进行二值化处理，利用八邻域算法巡线，而后根据左右边线的特征，识别赛道的情况，针对各种路段，给到下位机相应的指令。下位机在接收到指令后，通过 PID 控制器输出 PWM 波控制舵机偏转，进而控制车辆转向，同时根据不同目标速度给到不同的占空比，通过我们自己设计的驱动板，驱动有刷电机转动，进而带动后轮转动。另外，摄像头捕捉到的图像会直接传递给上位机中的 YOLO 模型，对图像中的内容进行识别，根据识别到的元素以及元素所在的位置，上位机进行不同的处理，进而控制车辆完成不同的操作。

整个开发流程如图 2.2 所示。上位机设计方面，小组采用 Vscode 进行 C++程序设计，利用 VNC 远程桌面对车辆的上位机进行控制，采用 WinSCP 辅助文件配置。下位机设计方面，小组采用 AURIX Development Studio 嵌入式开发软件对下位机进行 C 语言程序设计，对于下位机的参数调试，我们利用 Vofa 串口调试工具生成图像进行调参。AI 模型设计方面，小组采用百度飞桨的模型框架生成 YOLO-v3 模型，部署到远程工作空间中。硬件设计方面，小组采用嘉立创 EDA 软件进行 PCB 绘制，借助嘉立创平台打板。机械结构设计方面，小组利用 SOLIDWORKS 以及 CATIA 进行车壳结构设计，以及智能车中各个部分的零部件设计、整车运动仿真等。整车的代码我们通过 Git 进行协同管理。

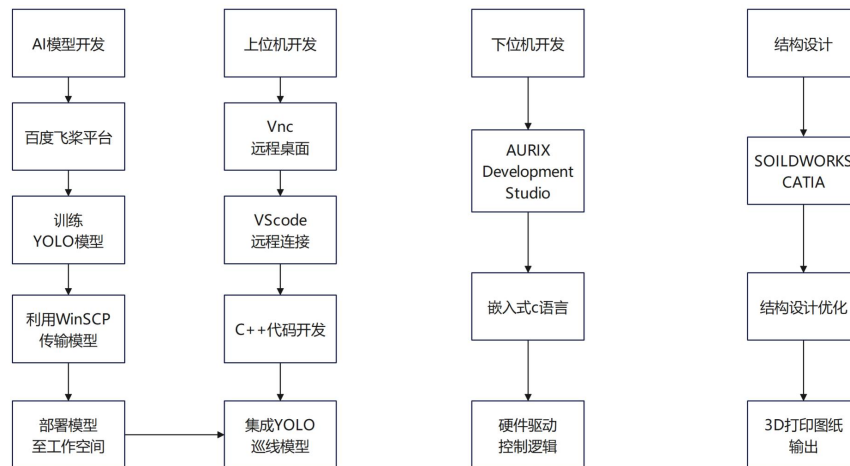


图 2.2 开发流程图

1. 关键技术、难点与对应的技术实施方案以及实验验证分析

3.1 八邻域算法设计与优化

3.1.1 八邻域算法的原理设计

为了获取到当前图像的信息，我们采用八邻域算法进行扫线处理。

首先，我们将摄像头感知到的图像进行二值化处理，得到一个二值化图像如图 3.1. 1 所示。

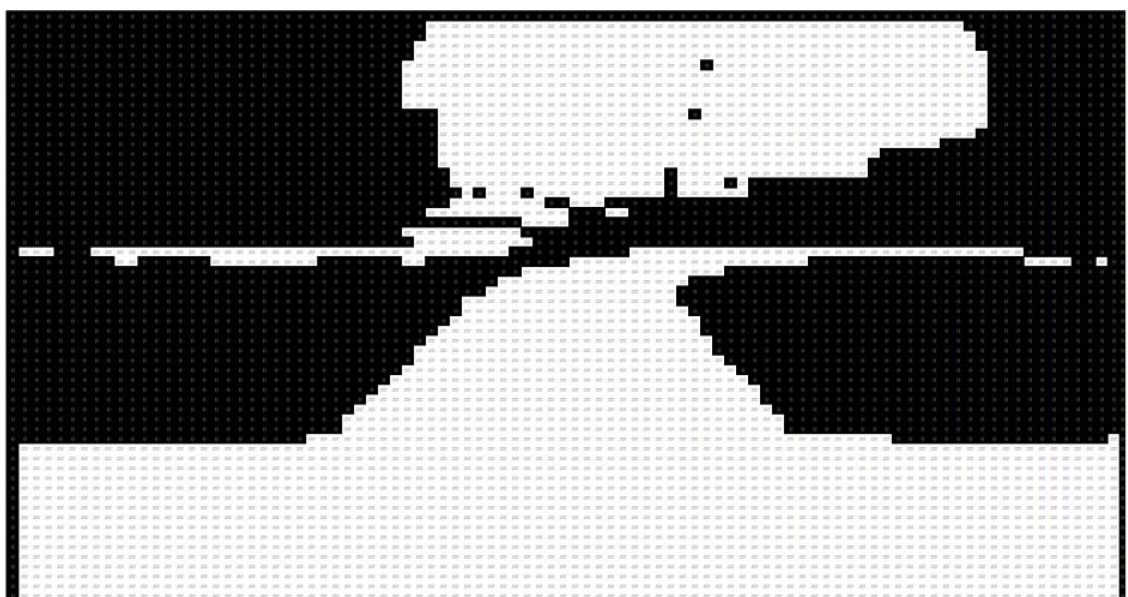


图 3.1.1 二值化图像

我们首先定义任意一个像素点周围的 8 个相邻区域为如下图 3.1.2 编码。存在顺时针编码与逆时针编码两种模式。此处以顺时针编码为例，并以左边线的获取为例。我们将从二值化图像的中间开始往左搜寻，找到白->黑的一组点作为八邻域搜寻的起点。获得起点后，以其为中心点，我们顺时针对其如图所示的邻域点进行顺时针遍历，当某个邻域点满足从黑点跳到白点的特征，就标记该点的编码作为该点的生长方向，并跳转到该点（黑点）作为中心点，再次进行该中心点的八邻域的遍历与检索，以此类推。

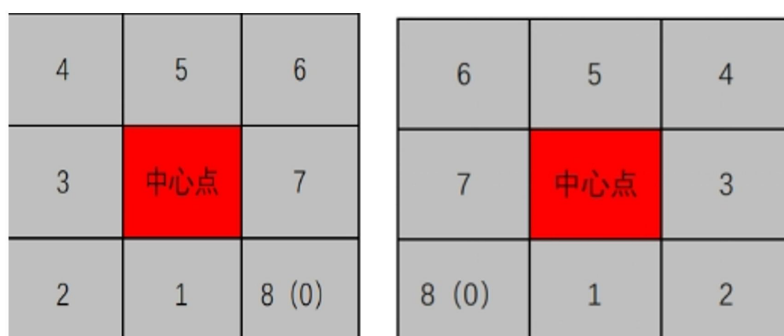


图 3.1.2 八邻域生长方向编码

例如，如图 3.1.3，二值化后将黑色标记为 0，白色标记为 255，假设当前中心点为红色框的正中心点，我们将顺时针遍历其八个邻域，发现绿色的点由黑到白（0 到 255），我们将中心点转为绿色的点，然后遍历绿色点的八个邻域，找到紫色点符合要求，跳转至紫色点作为中心点。



图 3.1.3 八邻域遍历过程实例

右边巡线类似，以逆时针编码。按上述类推从下往上循环便可以获得如下图 3.1.4 巡线结果。

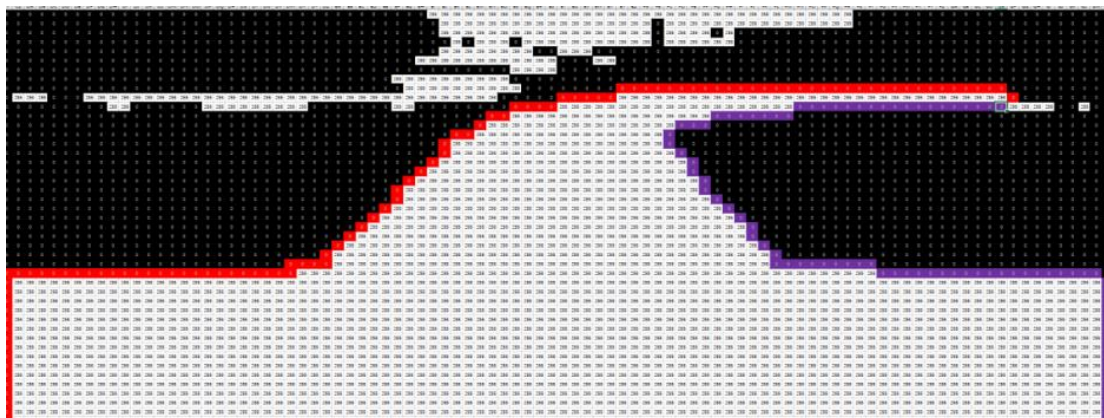


图 3.1.4 八邻域巡线结果示例

当边线反复回到某个点三次之后，继续按照上述方案巡线，会一直陷入循环内，为了防止程序卡死在当前阶段，我们在此时跳出循环，强行断开巡线，该边线巡线终止。若左右边线都没有出现上述情况，也就是的确都遍历完成，那么左右边线将会在二者相遇的地方掐断。

3.1.2 八邻域算法的进一步处理

在 findline.cpp 与 findline.hpp 中设计好了八邻域程序，并将所需的数据结构封装到了 Findline 类下。经过对二值化图像的处理，将会得到许多赛道的特征。我们将这些信息封装成为数组作为 Findline 的成员变量，为后续处理提供素材。

首先，我们经过对二值化图像的八邻域搜索，得到了左右赛道的点集，代码如下：

```
std::vector<cv::Point> left_point;           //图像左边原始点集

std::vector<cv::Point> right_point;          //图像右边原始点集
```

原始点集以 cv::Point 这种 opencv 数据结构进行存储，能够捕捉到八邻域算法所获取到的所有点的坐标。但这并不利于我们计算汽车需要行驶的路径。我们的图像整体是 240 行，320 列的二值化图像，我们的目的是希望获得中线点集，该点集以图像的行作为索引，储存点所在的列，这要求我们的中线的每一列只存在一个中线上的点。为此，我们需要遍历 left_point 以及 right_point，经过 C++ 代码处理使得每一行只存在一个点，该点的要求为，是同一行内所有的点中靠近赛道内侧的点。也就是要求左边点集取最右侧的点，右边点集取最左边的点。将所有的点组合为数组，代码如下：


```
std::array<int,240> leftpoint;           //处理后的左边点集

std::array<int,240> rightpoint;         //处理后的右边点集
```

该点集以行为索引，储存了赛道边缘的列。

接下来，我们对左右点集在同一行的列取中值，可以初步得到中线轨迹，储存在 `middleline` 数组中。

另外，之前提及的生长方向我们会储存在 `dir_l`，`dir_r` 数组中，该数组与 `left_point`，`right_point` 索引一致，存储在八邻域巡线运行过程中每一个点的方向编码。

3.1.3 八邻域算法在实际运行中的技术难点

在上位机程序设计方面。首先对于整个赛道的巡线就是一个较难处理的问题，传统的八邻域算法尽管可以完成绝大多数情况的巡线，但在实际运行的过程中，由于速度的影响，会出现赛道丢失，巡线遇到噪点而断开，甚至完全无法巡线等问题。

上述情况的问题所在主要是因为，在较小区域内，反复出现黑白噪点，使得巡线循环运行到此处的时候反复在图像这部分“打转”，为了使得循环跳出噪点，我们对八邻域进行了优化处理，传统八邻域只会检索中心点附近一圈的8个相邻点，我们尝试不再局限于附近一圈的邻域而是检索附近 n 圈的点。我们调试了这个 n 的值，太小的话无法跳出，太大的话，由于图像中可能存在其他赛道，这会使得八邻域算法检索到其他赛道上去。最终选择 $n=4$ 效果最佳。跳出噪点代码如下：

```
// ... 主循环开始 ...

我把他删掉了 // ... 主循环继续 ...
```

上述代码主要逻辑是生成方形轮廓。

我们采用了一种“方形扩张”的搜索策略。从卡死点 `errorPoint_l` 开始，先搜索最近的一圈 ($i=2$)，如果没找到合适的点，就扩大一圈 ($i=3$)，再扩大一圈 ($i=4$)。

圈数选择 ($i = 2; i \leq 4$):

$i=1$ (第一圈) 是八邻域本身。

i=2: 搜索距离卡死点 2 个像素远的圈。这个距离通常能跳过小的噪声块。

i=3, i=4: 继续扩大范围, 以应对更大的噪声区域。

代码没有填充整个方形区域, 而是通过四个循环分别生成了方形的四条边上的所有点。这避免了重复检查内部点 (那些点更可能是无效的噪声区域), 极大地提高了搜索效率。

第一个循环: 生成方形的右边线 (X 坐标固定为 $\text{errorPoint_l.x} + i$, Y 坐标从上到下扫描)。

第二个循环: 生成方形的上边线 (Y 坐标固定为 $\text{errorPoint_l.y} + i$, X 坐标从右到左扫描)。

第三个循环: 生成方形的左边线 (X 坐标固定为 $\text{errorPoint_l.x} - i$, Y 坐标从下到上扫描)。

第四个循环: 生成方形的下边线 (Y 坐标固定为 $\text{errorPoint_l.y} - i$, X 坐标从左到右扫描)。

3.1.4 八邻域算法的实验验证与分析

最终, 我们将代码梳理完成之后, 获得了循迹图, 如图 3.1.5。车辆能够正确处理二值化图像, 并利用八邻域算法获取到图像信息, 得到左右边线。我们将左右边线以及中线描绘在图像中, 以便观察。



图 3.1.5 八邻域巡线结果图

在实际运行过程中, 我们发现, 由于摄像头视角的问题, 车头会被投射到摄像头内部, 偶尔二值化效果较差的时候, 底部几乎全是黑色, 而八邻域算法若是在车头部分被噪点卡住, 那么就会影响整体巡线效果, 这会导致在车辆在较暗的环境下几乎无法运行。面对这样的情况, 我们选择将摄像头底

部图像切行处理，也就是使得底部左右边线的列均为默认值（5 和 315）分别底部的中线默认固定在图像正中央，这便可以防止错误巡线导致后续处理到错误信息，也可以防止车辆由于错误巡线而无法正常运行。暗光条件下切行前后巡线效果对比如图 3.1.6。

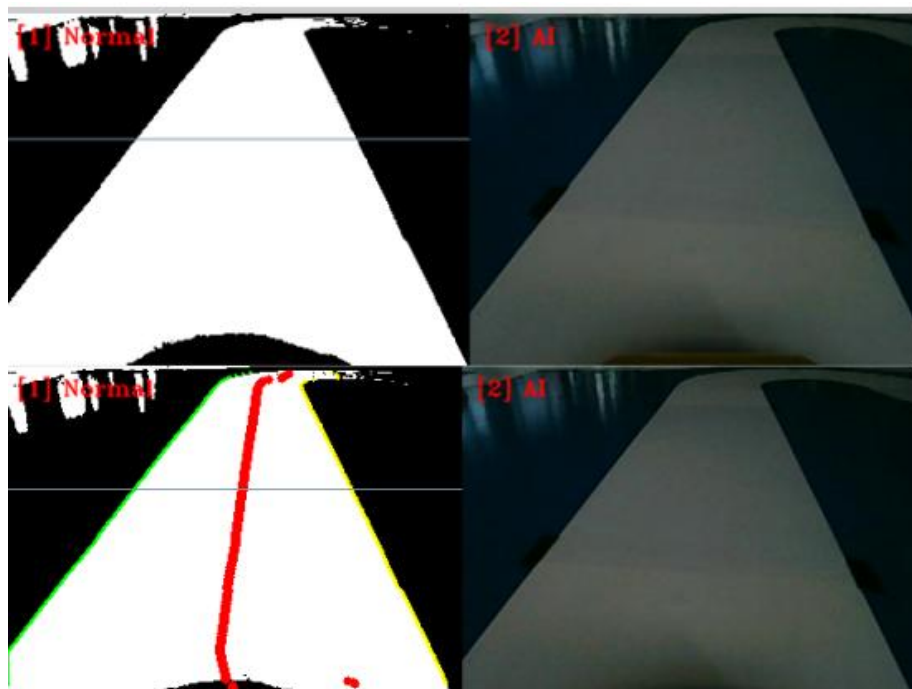


图 3.1.6 暗光条件下切行前后巡线效果对比

3.2 弯道处理的设计与优化

3.2.1 弯道处理的必要性

初期我们在调试的过程中发现，车辆一旦提高速度，在弯道中就极其容易冲出赛道，主要原因在于，图像中的赛道并非真实的赛道。实际情况是，如图 3.2.1 弯道示意图，由于摄像头是非广角摄像头，实际赛道会被截取掉一部分，这在弯道内体现明显。

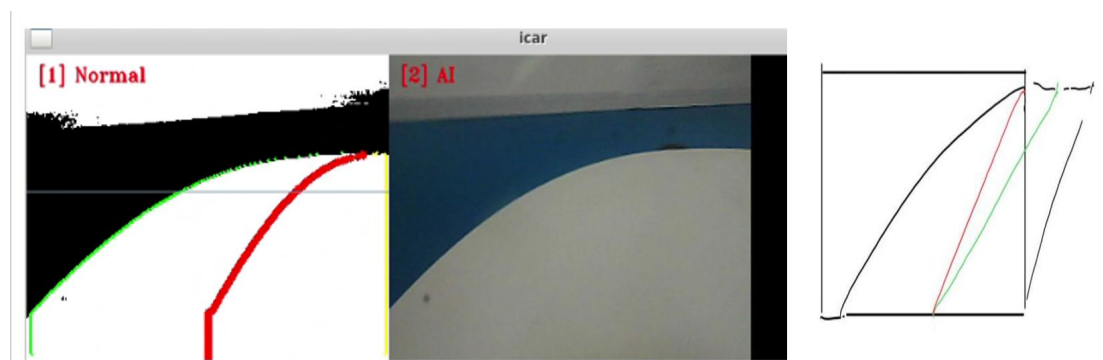


图 3.2.1 弯道示意图

以右拐弯为例，实际中线如图所示，会比当前的中线更向右偏移，也就是“拐少了”。

3.2.2 弯道识别

为了对弯道处理，首先需要识别弯道。同样以右拐弯为例，我们发现，赛道的左边线的生长方向存在特征，从下向上看，图像呈现向右上方向生长的趋势，实际代码中对应生长方向的编码为 5，6。而右边线由于丢线，生长方向竖直向上，对应编码为 4，或者靠近图像最右侧（此处以 310 为界限）。同时左、右侧边线交叉的点的行应该在图像的某行以下。思路流程如图 3.2.2。

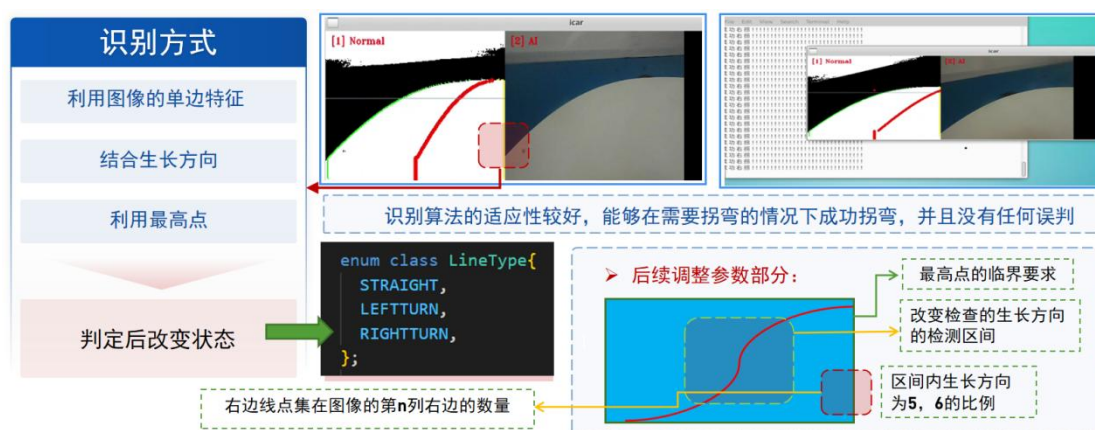


图 3.2.2 弯道处理思路流程

代码如下：

我把它删掉了

3.2.3 弯道处理

由于实际情况下，以右拐弯为例，在当前中线的基础上，中线应该向右偏移，所以我们需要对中线进行平移处理。

首先，我们要求最下方的点在平移前后都保持不变，而后每上移一行，将右边线向右额外平移一个单位。

但是我们发现上述方案效果并不理想，拐的太急容易靠内侧冲出赛道，太缓则容易向外冲出赛道。对此，根据不同的弯道急缓程度，我们还需要有一个线性化的处理，我们定义了一个偏率，加在右边线向右平移量的前面作

为一个系数。例如，左边的赛道相对竖直，那么应该拐得较少，偏率较小，左边的赛道相对水平，那么应该拐得较多，偏率较大，这点可以利用左边赛道的斜率刻画。再者，左右边线的交叉点的行也是一个刻画标准，交叉点在图像下方，显然弯道更急，而在图像上方，显然弯道更缓。两个因素使得我们开始纠结方案的选取，单独选择任何一个标准都会显得考虑不够周全。

最终，我们考虑到，一条线应该存在两个自由度，任意两个因素都可以确定一条直线（近似把左边线视作直线），因此，如图 3.2.3，我们选择了横、纵截距 B_x , B_y 作为刻画因素。

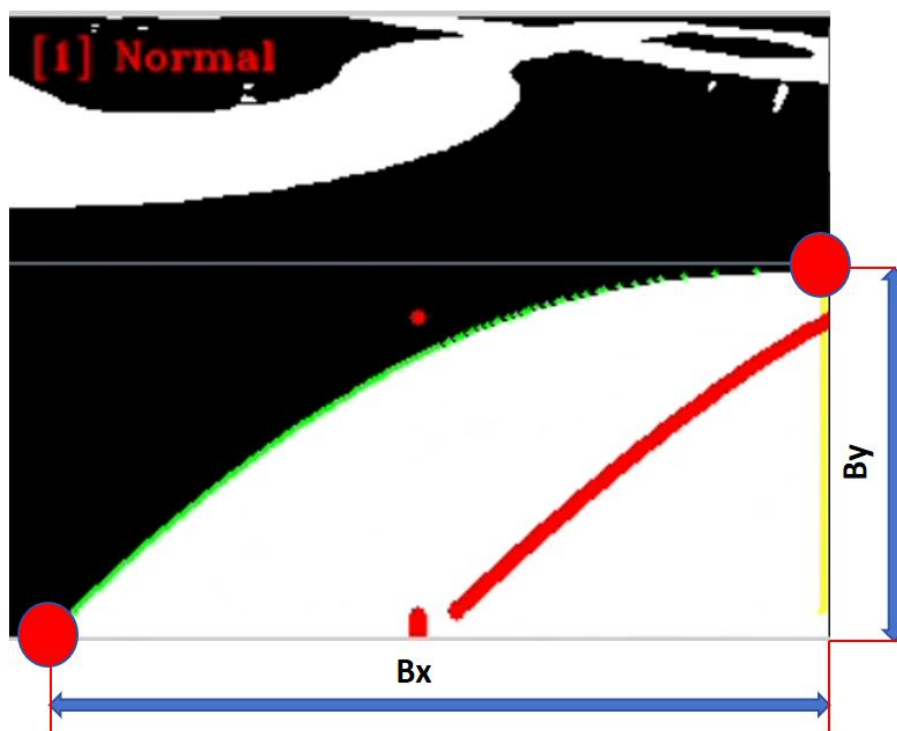


图 3.2.3 横纵截距示意图

而为了使得横、纵截距的地位相当（考虑到图像是 240×320 的规格，令 $X=320$, $Y=240$ ），我们将横纵截距进行了归一化处理，使得二者具备可比性。然后取二者相对较小的一个作为标准：

$$B = \min (B_x/X, B_y/Y) \quad (1)$$

而后将 B 与一个标准值 $slopeConst$ 作差比对，如果大于这个值，那么认为当前中线需要偏移处理，如果小于，那么认为不需要偏移。这主要是为了防止一旦识别成功为弯道，那么车辆立即拐弯，会使得车辆由于突变的舵机打角而晃动，所以我们在识别成功拐弯状态之后，会留下较小的阈值，这部分情况不进行偏移处理，而后进行线性偏移，以防在一些临界的情况下，车辆会出现一会识别成功弯道，一会识别失败的情况，这使得车辆发生来回

晃动。我们令这个值为 ΔB :

$$\Delta B = \max(B - \text{slopeConst}, 0) \quad (2)$$

再乘以一个系数 slopeRate , 求得右偏率 right_stdio :

$$\text{right_stdio} = \text{slopeRate} * \max(\min(Bx/X, By/Y) - \text{slopeConst}, 0) \quad (3)$$

令某一行 y 的左边线的列为 X_{left} , 右边线为 X_{right} , 计算得补偿后的第 Y 行的中线 midline_y 为:

$$\text{midline}_y = (X_{\text{left}} + X_{\text{right}} + \text{slopeRate} * \max(\min(Bx/X, By/Y) - \text{slopeConst}, 0)) * (Y - y) / 2 \quad (4)$$

同理, 左拐弯的时候, midline_y 为:

$$\text{midline}_y = (X_{\text{left}} + X_{\text{right}} - \text{slopeRate} * \max(\min(Bx/X, By/Y) - \text{slopeConst}, 0)) * (Y - y) / 2 \quad (5)$$

对应代码如下:

```
...直道处理...  
  
//左拐弯midline 计算  
  
你们自己把理论改成代码吧
```

3.2.4 弯道处理的实验验证与分析

最终调试发现, 常规弯道下拐弯的效果较好, 较好地完成了巡线。巡线前后对比如图 3.2.4。

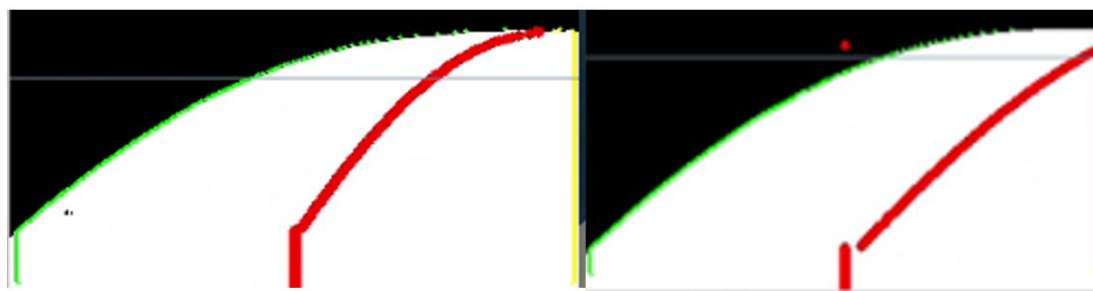


图 3.2.4 弯道巡线前后对比

但考虑到在连续 S 弯的衔接处，会有一部分弯道无法被识别为弯道。

这主要是因为，左右弯道衔接的时候，会留白一部分，导致左右边线无法交叉，针对这种情况，我们将八邻域算法中左右边线断开的条件进行了修改，将交叉这一条件修改为了左右边线相对靠近时便掐断。最终，车辆在 S 弯中便可以较为顺畅地完成拐弯。最终效果前后对比如图 3.2.5。

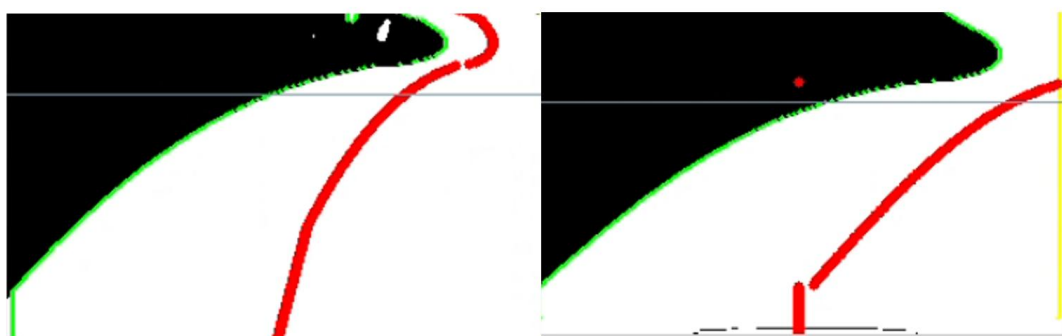


图 3.2.5 S 弯前后处理对比

最终，调试得到实际车辆在弯道内圈、外圈以及正常道路上的舵机打角如图 3.2.6。可以发现，在弯道内圈车辆拐弯幅度小（舵机打角小），在弯道外圈拐弯幅度大（舵机打角大），正常道路上只有小幅度拐弯甚至没有拐弯。这也符合实际车辆的运行效果。



图 3.2.6 舵机打角效果对比

3.3 十字路口的算法设计

3.3.1 十字路口的处理必要性

十字路口的参考图如图 3.3.1 所示。汽车需要进入十字后绕行出十字。

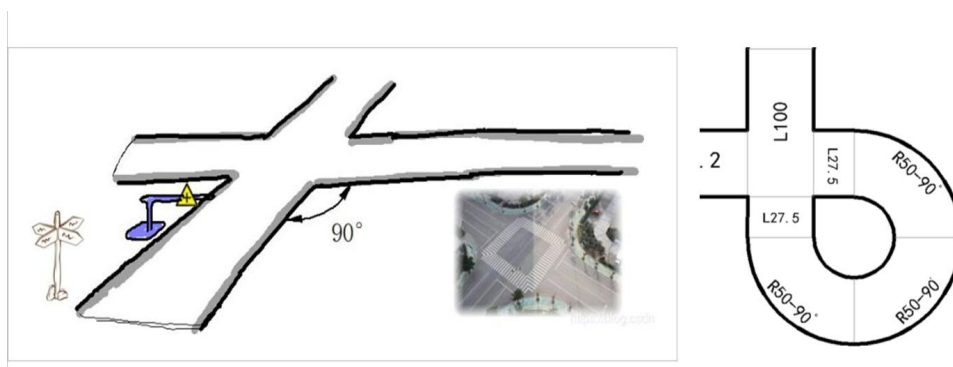


图 3.3.1 十字路口示意图

常规正入十字的时候，其实中线几乎还是在图像正中间，故正入十字并不需要处理。而面对拐弯之后进入十字路口，也就是斜入十字，由于八邻域巡线的弊端，以及非广角摄像头的信息丢失，导致车辆遇到十字时会轨迹会偏移，甚至直接忽略十字路口。例如如图 3.3.2 的一个右斜入十字的情况，我们发现整个赛道的右边线扫描到了赛道的左边，导致整个中线呈现一个向左拐弯的趋势。所以，这需要对斜十字路口进行特殊识别，并做出一定的处理。

3.3.2 十字路口的识别

以右斜入十字为例，观察图 3.3.2 可知，这种情况与一个弯道中间“伸出一只手”非常相似，因此，我们需要刻画这样的一个特征。



图 3.3.2 十字路口特征图

首先，我们需要“找点”，对于这样的图像，存在三个特征点，如图 3.3.3。第一，当我们把“伸出的那只手”去掉，那么剩下的就是一个弯道，我们需要找到弯道的最高点，该点的特征是往前的点的生长方向都是向上的或者在图像相对靠右的位置，而后的点的生长方向均向左，我们令该点为 cross_endline。第二，我们把十字的左上角点作为特征点，该点的特征主

要是往前的若干点生长方向为左下，往后若干点朝左，我们令该点为 cross_back。第三，左边线上的十字的左下角点，该点的特征为往前若干点生长方向朝左上，往后若干点朝左，我们令该点为 cross_down。

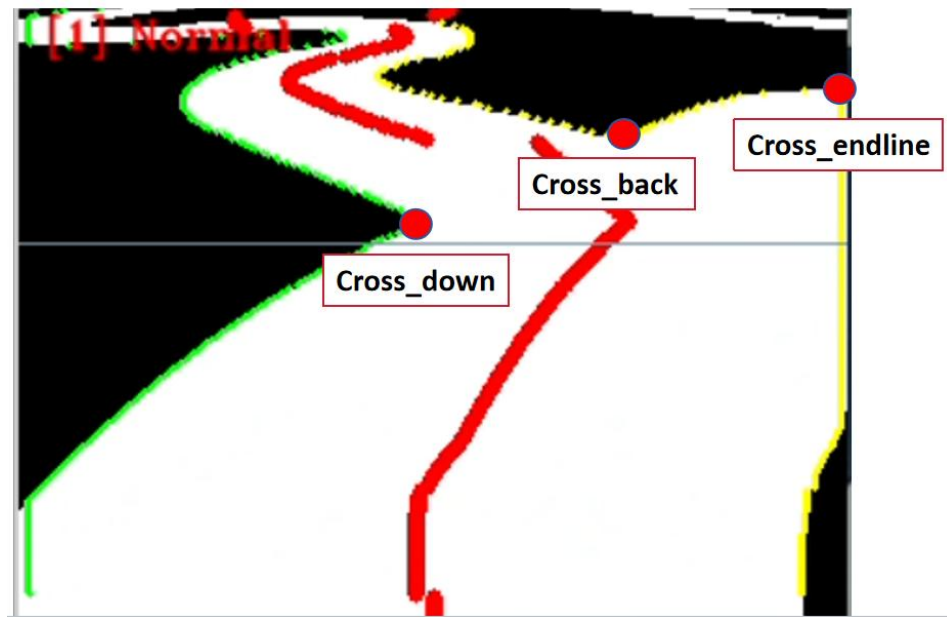


图 3.3.3 十字路口特征点示意图

以第一个特征点为例，“找点”的代码如下所示：

```
// Step 1: 查找第一个特征点 (使用 dir_str 和 str_point)

for (int i = current_corner; i < dir_str.size(); i++) {

    int flag = 1;

    int correctCount = 0;

    for (int j = i; j < i + 10; j++) {

        int count = std::min(j, (int)dir_str.size() - 1);

        if (dir_str[count] != 4 || (flag_left && str_point[count].x > 10) || (flag_right && str_point[count].x < 310)){

            correctCount++;

            // std::cout<<"count: "<<count<<" dir_str[count]: "<<dir_str[count]<<" str_point[count].x: "<<str_point[count].x<<std::endl;

        }

    }

    if (correctCount > 3) flag = 0;

}
```

```

        correctCount = 0;

        for (int j = i + 20; j < i + 50; j++) {

            int count = std::min(j, (int)dir_str.size() - 1);

            if (dir_str[count] != 6 && dir_str[count] != 7 && dir_str[count] != 5 && dir_str[count] != 0) {

                correctCount++;

                // std::cout<<"count: "<<count<<"  dir_str[count]: "<<dir_str[count]<<std::endl;

            }

        }

        if (correctCount > 3) flag = 0;

        if (flag == 1) {

            step1 = 1;

            step1_count = i + 25;

            cross_end_line = str_point[step1_count];

            current_corner = i + 25;

            break;

        }

    }
}

```

此处代码大意为，从下往上遍历右边线，对于每一个点，检索其前方 10 个点，若其中有 3 个点不满足要求，那么该点舍弃，再检查后方第 10 到第 50 个点，若超过 3 个点不满足要求，那么该点舍弃。最终可以筛选出第一个特征点。而后的第二个特征点的获取，可以从第一个点出发开始遍历右边线。

将三个特征点找到之后，接下来就需要描述特征点之间的点的特征了。基于之前提及的这部分图像比较类似于弯道“伸出一只手”的形状，我们可以描述出以下特征。

首先，弯道特征，我们可以看到，右边线一直延申到 cross_back 特征点之后，显然，cross_endline 到 cross_back 中间的点，加上左边线中，在 cross_down 之前的点，这些点具备弯道的特征，只需检测这些点的生长

方向即可。

除此之外，对于 `cross_down` 与 `cross_back` 之间也存在约束。这两个特征点是十字路口在同一侧的上下角点，所以一方面，二者的 `y` 值应该有一定差距，而 `x` 值差得相对较小。观察图 3.3.4 发现，令左边线的出发点为 A，`cross_down` 为 O，`cross_back` 为 B， $\angle AOB$ 的大小一定相对较大，为此，我们需要对其余弦值进行约束。

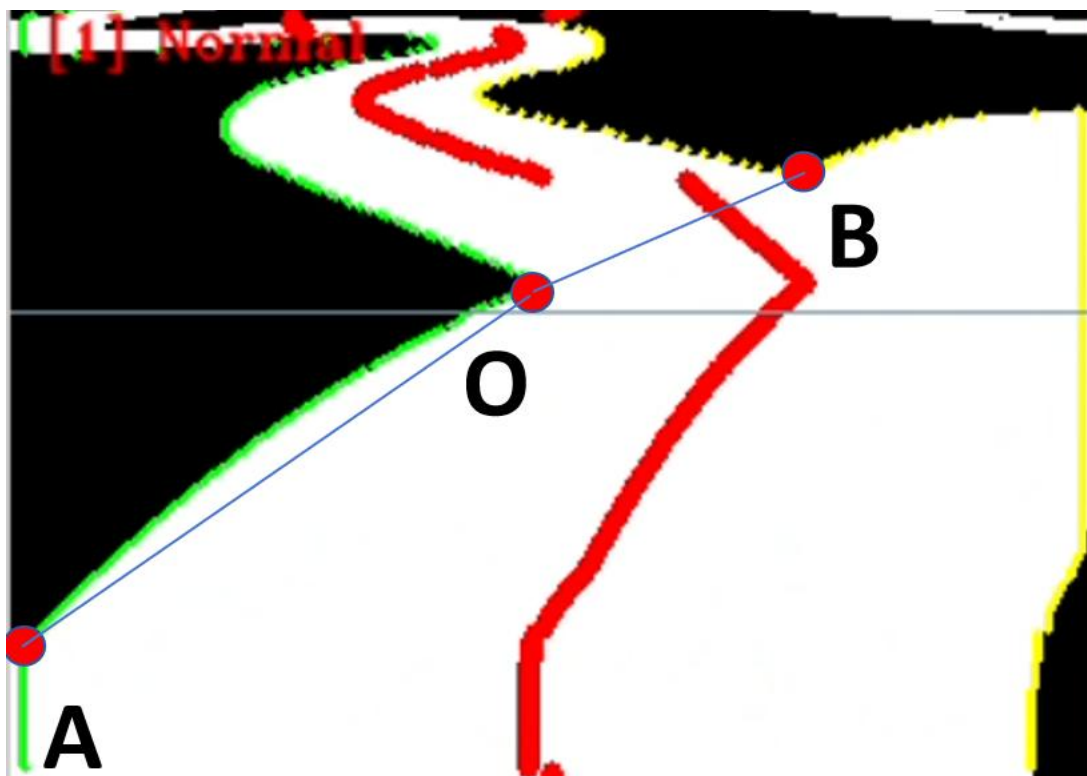


图 3.3.4 十字路口中的特征角度

代码如下：

```
double Cross::calculateCosineAngle(const cv::Point& vertex,
                                   const cv::Point& point1,
                                   const cv::Point& point2) {
    // 创建两个向量
    cv::Point2f vec1 = cv::Point2f(point1) - cv::Point2f(vertex);
    cv::Point2f vec2 = cv::Point2f(point2) - cv::Point2f(vertex);

    // 检查零向量
```

```

if ((vec1.x == 0 && vec1.y == 0) || (vec2.x == 0 && vec2.y == 0)) {

    std::cerr << "错误:零向量!点重合或无效点" << std::endl;

    return 1.0; // 返回无效值

}

// 计算点积

double dotProduct = vec1.x * vec2.x + vec1.y * vec2.y;

// 计算向量模长

double magnitude1 = std::sqrt(vec1.x * vec1.x + vec1.y * vec1.y);

double magnitude2 = std::sqrt(vec2.x * vec2.x + vec2.y * vec2.y);

// 计算余弦值

double cosine = dotProduct / (magnitude1 * magnitude2);

// 处理浮点精度问题, 确保结果在[-1, 1]范围内

if (cosine > 1.0) cosine = 1.0;

else if (cosine < -1.0) cosine = -1.0;

return cosine;

}

```

最后，需要根据左右十字的约束进行一个方向约束，例如，cross_endline 特征点一定在右侧（斜入右十字）。这些约束相对简单就不多赘述。

3.3.3 十字路口的补线处理

从理论上讲，这部分的正常巡线应该是将 cross_endline 之后的所有的右边线全部给到左边线，并且连接 cross_down 与 cross_back，按照之前八邻域后续处理的规则，实际的轨迹就会按照目标的情况规划。上述内容的代

码实现逻辑为，在 while 循环内，从右边线点集的最后开始将点“扔出来”，然后放到左边线点集中，只要当前点的 x 值没有到最右边附近，就一直在循环内。而后利用线性插值将 cross_down 与 cross_back 连接起来，放入左边线点集内。代码如下：

```
void Cross::Cross_repair_right(Findline &findline){  
  
    while(!findline.pointsEdgeLeft.empty() && findline.pointsEdgeLeft.back().col >10) {  
  
        findline.pointsEdgeRight.emplace_back(findline.pointsEdgeLeft.back());  
  
        findline.pointsEdgeLeft.pop_back();  
  
    }  
  
    findline.endline_eight = findline.pointsEdgeRight.back().row + 1;  
  
    if(cross_back != cv::Point{0,0} && cross_down_point != cv::Point{0,0} ){  
  
        int rowStart = cross_down_point.y;  
  
        int rowEnd    = cross_back.y;  
  
        int colStart = cross_down_point.x;  
  
        int colEnd    = cross_back.x;  
  
        if (rowStart <= rowEnd){  
  
            return;  
  
        }  
  
        for (int rowCur = rowStart; rowCur >= rowEnd; --rowCur){  
  
            float t = float(rowCur - rowStart) / float(rowEnd - rowStart);  
  
            float colInterp = colStart + t * (colEnd - colStart);  
  
            int final_row = rowCur;  
  
            int final_col = static_cast<int>(colInterp);  
  
            findline.pointsEdgeRight.emplace_back(mpoint( final_row , final_col));  
  
        }  
  
    }  
  
}
```


3.3.4 十字路口处理的实验验证与分析

最终，在十字路口出入口处，完成补线如图 3.3.5。

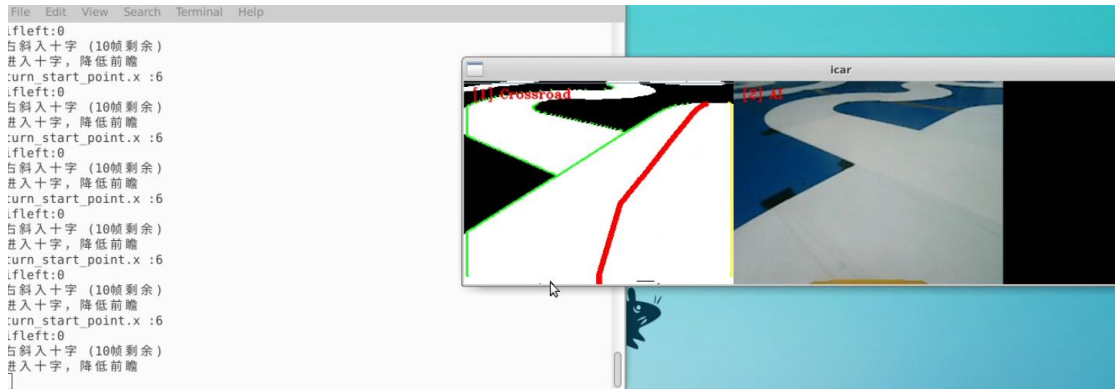


图 3.3.5 十字路口补线效果

但是，我们发现，十字路口的识别方面，存在一定意义上鲁棒性与灵敏度的矛盾。我们很难做到识别到任意视角的十字路口（因为从设计目的上来看就是解决极端视角下的情况），一旦放宽条件，及其任意误判。对此，我们采用了“保持”策略。我们将条件设定相对苛刻，目前能够做到出入十字必定会有至少一次成功识别到十字。我们设定了一个十字状态，一旦成功识别十字，则进入十字状态，这个状态会保持若干帧，在这个状态内，只要能够找到 **cross_endline**, **cross_back**, **cross_down** 三个关键节点，那么无论节点之间是否满足十字的要求，都会进行补线操作（将一侧的线给予另外一侧，并且连接 **cross_back**, **cross_down**）。

经过这样的处理，使得车辆在出入十字路口时，更加顺畅，不会出现之前向一侧偏移的情况。

3.4 临时停车区的设计与优化

3.4.1 临时停车区的大体任务

在临时停车区会有一个指示牌，指示牌的内容为学校或者公司。车辆在识别到临时停车区标识牌后，需要驶入停车区域（如图虚线框内）停车片刻，模拟真实车辆停车接送乘客。标识牌一般放置在停车区的同侧，具体放置要求如图 3.4.1。

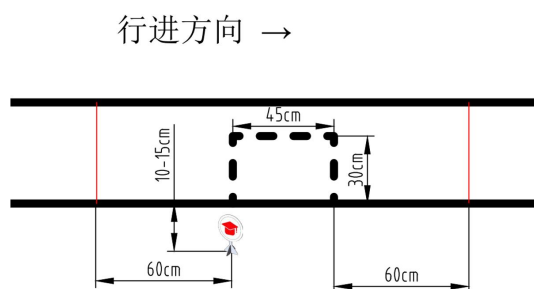


图 3.4.1 临时停车区示意图

3.4.2 临时停车区方案迭代过程

初期，我们的思路是识别到标识牌之后打一个固定角度，等到标识牌消失之后若干帧，开始停车。这样的方案几乎是开环完成任务，一旦车辆的姿态发生改变，效果就大打折扣。为此，我们需要采取闭环的方案，尽可能地用上赛道上的内容。

为了使得车辆能够在前后方向停稳，我们采取了扫描黑线的方案，在标识牌消失若干帧后，进入扫描状态，此时如若扫描到前方黑色像素超过了某个阈值，那么就进入停车状态。

为了使得车辆在左右方向可以停稳，我们将虚线框作为了 AI 识别的对象，一旦识别到虚线框，由于 YOLO 会反馈给图像一个矩形区域，将虚线框包裹起来，我们选择将虚线框的一侧的列作为边线的列，使得整个车辆的轨迹“对准”虚线框，以保证车辆能够恰好停稳在虚线框内。

最终，根据上述方案迭代，我们设计了如下的状态机：Temporary。该状态机分为如下几个状态：

```
enum class TemporaryStep
{
    None,
    Enable,
    Left,
    Right,
    Out,
    Scan,
    Stop
};
```

在 None 状态，车辆会对标识牌进行检测，一旦识别到标识牌满足在某个距离内，并且识别到若干帧，则进入 Enable 状态。如若发现第一次识别到标识牌时以及特别近了，那么车辆进入 Out 状态。

在 Enable 状态，车辆会对标识牌的位置进行识别，若标识牌在赛道左侧，接下来进入 Left 状态，在右侧，接下来进入 Right 状态。并且对车辆进行减速处理。

在 Left 状态，车辆会识别虚线框，此时会将赛道右边线替换为虚线框的右边线，使得中线轨迹几乎对准虚线框中心。（Right 同理）

在 Out 状态，由于此时虚线框距离车辆相对较近，虚线框对于车辆来说是几乎正视的状态，我们选择直接将左右边线全部替换为虚线框的左右边线。

对于 Left, Right, Out 三个状态几乎是平行的地位。当标识牌消失若干帧后，会进入 Scan 状态，此时进一步减速，并对黑色像素进行扫描。考虑到车辆的姿态问题，很可能车辆的姿态并未摆正，这导致如果仅仅扫描图像中某一行的黑色像素，会因为姿态问题导致即使虚线在面前也无法扫描成功。而如果选择利用矩形框区域进行扫描，可以避免这个问题。二者方案差异如图 3.4.2。

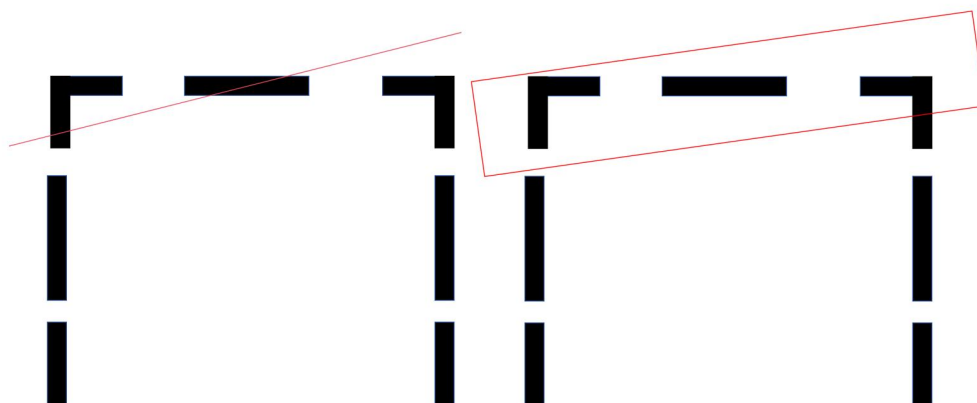


图 3.4.2 两种扫描方案的对比图

而后进入 Stop 状态，此时车辆速度设置为 0。若干帧后退出并重新恢复速度。

最终状态机的流程图如图 3.4.3。

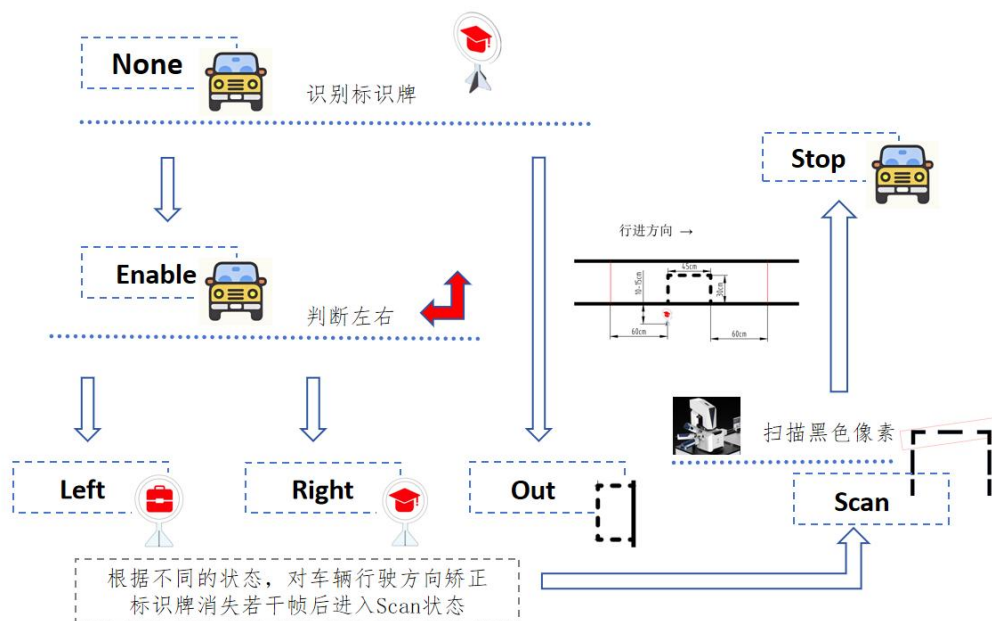


图 3.4.3 临时停车区状态机流程图

3.4.3 临时停车区路径的实验验证及分析

在迭代过程中，我们也在反复调试。在最终代码敲定后，得到较好的调试结果。部分调试结果需要视频展示，此处不便于视频展示，只能以图片形式完成一部分的实验验证。

经过验证，在较远距离时（Left/Right 状态），路径矫正如图 3.4.4 所示。

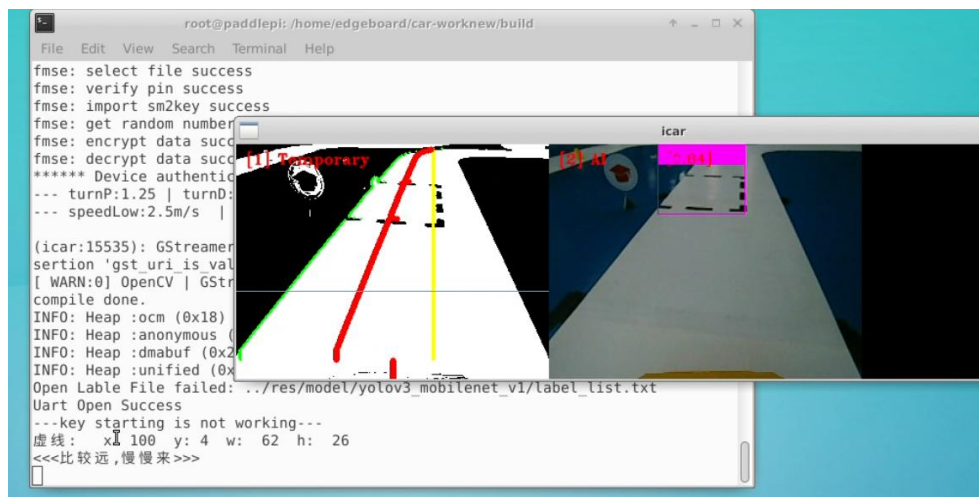


图 3.4.4 Left/Right 状态的补线方案

在较近距离时，路径矫正如图 3.4.5 所示。

实际车辆在临时停车区内识别到虚线框也的确会进行拐弯处理，如图 3.4.7。可以发现，车辆在识别到虚线框后，会根据虚线框的位置对舵机打角动态调整，例如图中车辆在左右位置固定的条件下，距离虚线框较远的时候，打角较小，较近的时候打角较大。一方面这符合预期效果，车辆根据虚线框的位置进行闭环处理，另外一方面这也符合实际车辆拐弯情况。

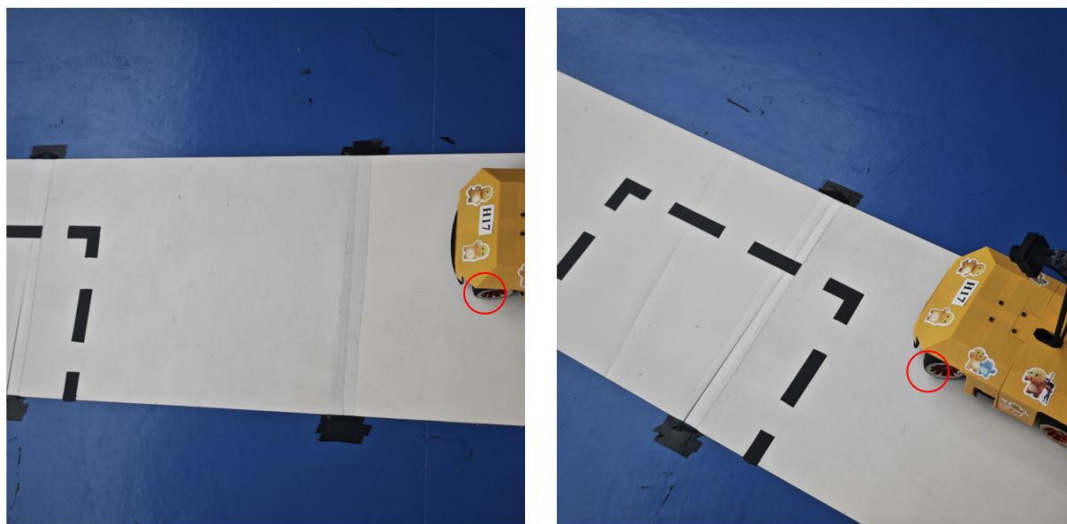


图 3.4.7 临时停车区内识别到虚线框后车辆的拐弯效果

总的来说，对于自动驾驶车辆来说，对于场景的适应性相当重要，我们无法预测实际情况下面需要面对的问题，所以，这要求我们针对实际场景给出一个可以自适应的普适性控制方案。

3.5 基于三阶贝塞尔曲线的避障方案

3.5.1 基于三阶贝塞尔曲线的避障方案的建模与实施

在直道上会随机摆放行人与锥桶，车辆需要针对摆放的位置进行避障，达到正常行驶的基础上并且不会与行人与锥桶相撞的目的。

考虑到行人与锥桶在模型化后是类似的，二者可以类似处理，只需要具体调参即可。想要实现现实中车辆绕行障碍的模式，大致绕行方案应当如图 3.5.1 所示。

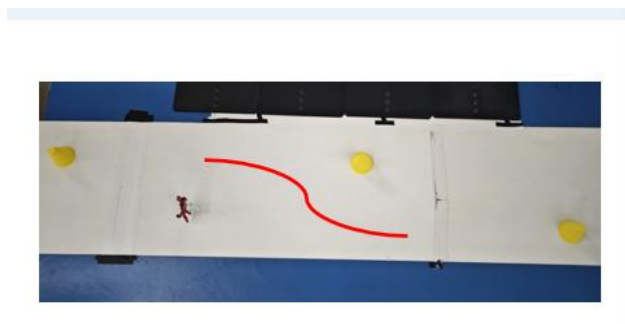


图 3.5.1 绕行方案示意图

我们根据 Yolo 模型所反馈的锥桶或是行人的位置信息，可以得到它们的坐标。通过查找资料以及上网搜寻相关开源代码，我们选择采用三阶贝塞尔曲线对避障路径进行拟合。贝塞尔曲线旨在利用已知的定点进行插值获得目标轨迹。三阶贝塞尔曲线的表达式如下：

$$B(t) = (1-t)^3 * P_0 + 3t(1-t)^2 * P_1 + 3(1-t)t^2 * P_2 + t^3 * P_3 \quad (6)$$

一般的贝塞尔曲线可以有如下的表达方式：

$$B(t) = \sum_{i=0}^n P_i * B_{i,n}(t), t \in [0, 1] \quad (7)$$

其中 $B_{i,n}(t)$ 是伯恩斯坦基多项式：

$$B_{i,n}(t) = \binom{n}{i} * t^i * (1-t)^{n-i} \quad (8)$$

从线性代数的角度来看，伯恩斯坦基多项式构成了 n 次多项式空间的一组基底，任何 n 次参数曲线都可以用这组基底和控制点来表示。

借由此形式，我们可以较轻松地给出 C++ 的代码区表达上述曲线，此处我们将曲线坐标化，分别计算了 x , y 两个方向的表达式。嵌套了两层循环，内层循环计算贝塞尔曲线表达式，外层循环按照 dt 步长计算离散化的各个 t 值下的曲线值，并存放至数组内。代码如下：

```
struct mpoint {
    double row;
    double col;
};

long factorial(int n) {
    if (n <= 1) return 1;
```

```

        return n * factorial(n - 1);
    }

vector<mpoint> Bezier(double dt, vector<mpoint> input) {

    vector<mpoint> output; // 输出点集

    double t = 0.0;        // 初始化参数 t

    int n = input.size() - 1;

    // 主循环:遍历参数 t 从 0 到 1

    while (t <= 1.0) {

        mpoint p;           // 当前 t 对应的曲线点

        double x_sum = 0.0;

        double y_sum = 0.0;

        int i = 0;

        // 内循环:对每个控制点计算伯恩斯坦基函数并加权求和

        while (i <= n) {

            // 1. 计算二项式系数  $C(n, i) = n! / (i! * (n-i)!)$ 

            double binom_coeff = (double)factorial(n) / (factorial(i) * factorial(n - i));

            // 2. 计算伯恩斯坦基函数值  $B_{\{i,n\}}(t) = C(n,i) * t^i * (1-t)^{(n-i)}$ 

            double bernstein = binom_coeff * pow(t, i) * pow(1 - t, n - i);

            // 3. 将当前控制点的坐标乘以基函数值, 并累加到总和

            x_sum += bernstein * input[i].row;

            y_sum += bernstein * input[i].col;

            i++;

        }

        // 将计算得到的点存入输出向量

        p.row = x_sum;

        p.col = y_sum;

        output.push_back(p);
    }
}

```

```

        // 增加参数t, 步进 dt

        t += dt;

    }

    return output;

}

```

表达式的意义在于在已知坐标系下, 根据上述定点将函数表达式表示出来, 进而生成曲线。

C++代码如下:

```

// 控制点1:当前赛道右边线中点
points[0] = track.pointsEdgeRight[row / 2];

// 控制点2:障碍物左侧下方位置

points[1] = {

    nearest_object.y + nearest_object.height,

    max(nearest_object.x - 2.5*nearest_object.width - ai2_params->roadway_angle_size_person, double(0))

};

// 控制点3:障碍物左侧上方位置

points[2] = {

    (nearest_object.y + nearest_object.height + nearest_object.y) / 2,

    max(nearest_object.x - 2*nearest_object.width - ai2_params->roadway_angle_size_person, 0)

};

// 控制点4:根据障碍物位置选择终点

if (nearest_object.y > track.pointsEdgeRight.back().row)

    points[3] = {

        track.pointsEdgeRight.back().row,

        track.pointsEdgeRight.back().col - ai2_params->roadway_angle_size_person

    }; //图像最下方的附近点

```

```

else

points[3] = {nearest_object.y, nearest_object.x}; //障碍所在位置

```

3.5.2 基于三阶贝塞尔曲线的避障的实验验证与分析

通过编写代码，完成了贝塞尔曲线的描绘，如图 3.5.3 所示为贝塞尔曲线描绘效果图。



图 3.5.3 贝塞尔曲线描绘效果

而后我们发现，行人与锥桶的重心不同，以及行人实际情况有可能张开手臂，这导致车辆一旦撞到行人，及其可能导致行人摔倒，而锥桶一方面被撞到的可能小，另一方面即使被撞到，也不一定会有明显移动。所以我们在锥桶与行人的所取定点的坐标中加入了补偿项，这部分补偿项 `roadway_angle_size_person` 与它们的宽度是同号的，这相当于给到二者一个等效宽度作为补偿。行人与锥桶的等效宽度有所不同，这个参数是需要调试获取的。

另外，我们发现当车辆恰好避过了一个障碍，此时车身与障碍同行的时候，接下来如若再遇到一个障碍，车辆会试图再次避障，此时车辆极有可能为了避开接下来的障碍而侧身撞击同行的障碍。如图 3.5.4 所示。为了避免这种情况的发生，我们设定并调试了一个参数 `roadway_pingbi` 对行数较小的（相对车辆较远的）障碍屏蔽，该参数将整个图像的上方 1/5 的区域的障碍进行了屏蔽处理，使得车辆在整个车身几乎越过前一个障碍后，才会设法避过后一个障碍。

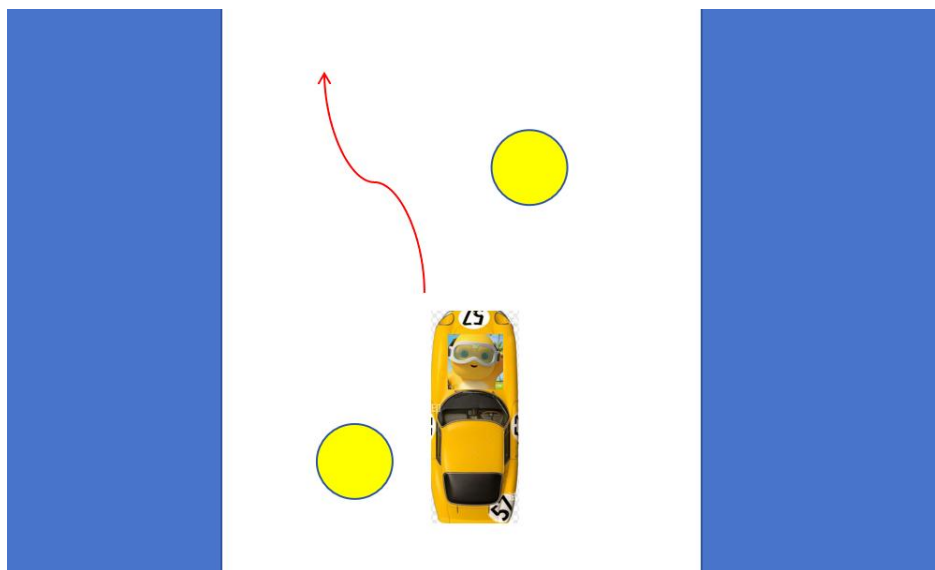


图 3.5.4 相邻障碍示意图

通过调试，最终我们获得了一个既能防止相邻障碍导致的相撞问题又能够保证车辆可以做到及时避障，最终效果如图 3.5.5 所示。可以发现，在车辆刚刚完成前一个障碍的避障时，后一个障碍的出现并不会导致车辆重新规划路线，这符合预期效果。

观察实际车辆的底盘，如图 3.5.6，在常规路障前方，车辆存在一个轻微幅度的拐弯，在整个车身仍旧与上一个障碍同行的情况下遇到了第二个障碍时，车辆不会拐弯，这符合我们对障碍屏蔽的设计要求。

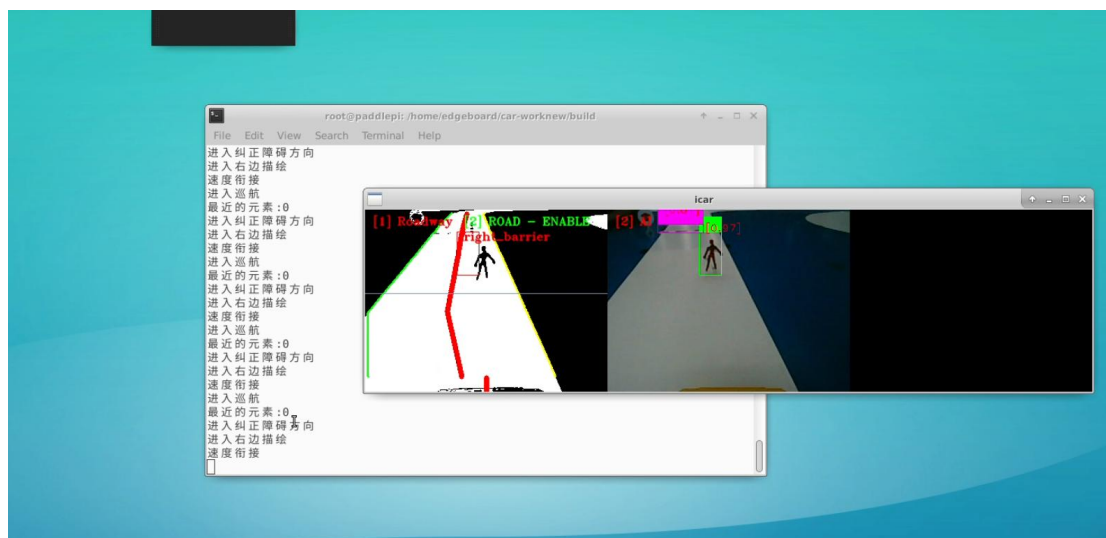


图 3.5.5 屏蔽处理后的效果图

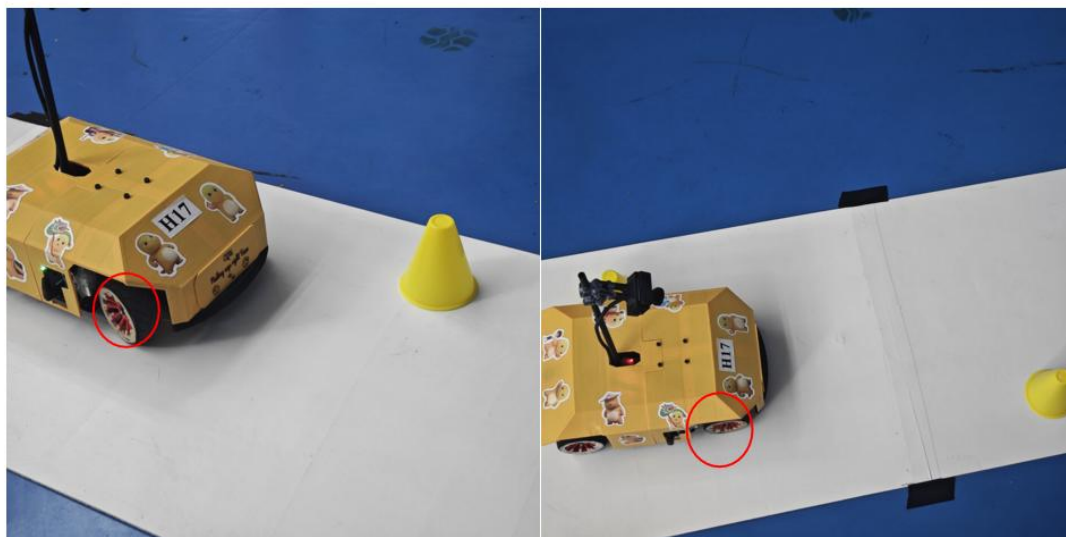


图 3.5.6 实际车辆避障效果

3.6 底盘 PID 的调参

3.6.1 底盘 PID 的重要性

实际运行时发现车辆的底盘反馈不够及时，面对需要完成急刹车，过弯减速的需求时，底盘很难完成。发现主要原因在于 PID 的参数不合适。

3.6.2 基于位置式 PI 控制器的电机闭环控制及其整定与实验验证

如图 3.6.1 为自动控制原理教材中针对智能车底盘控制的 PI 算法表达式解析。

数字 PI 算法如下

$$U_c = u_k = K_p e_k + K_I \sum_{j=0}^k e_j$$

其中： k ——采样序号， $k=0, 1, 2, \dots$ ；
 u_k ——第 k 次采样时刻的计算机输出值；
 e_k ——第 k 次采样时刻输入的偏差值；
 K_p ——比例控制系数；
 K_I ——积分控制系数。

图 3.6.1 自动控制原理中对 PI 算法的解析

根据教材所提及，PI 控制器的参数整定方案如下。首先只调整比例部分，将比例系数由小变大，直到响应快，超调存在（但并不算很大）为止。但实际发现，无论如何，智能车的速度仍旧与设定速度存在偏差，原因在于积累的误差，故需要逐步增大 I 值以消除稳态误差。整定过程利用 vofa+ 虚拟示

波器显示速度曲线，得到参数为： $K_p = 1.15$ ， $K_i = 0.215$ 。Vofa+图像如图 3.6.2 所示。

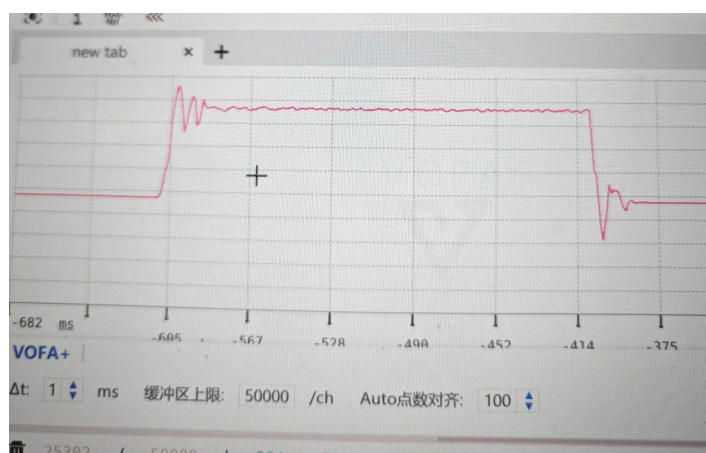


图 3.6.2 底盘 PID 整定结果图

最终，我们将底盘 PID 整定之后的车辆的驱动打开，设定速度为 0，并静止放置于坡道上方，如图 3.6.3。可以发现，无论是上坡还是下坡，车辆都可以稳定停止在坡道上方，这说明我们的 PID 整定方案符合预期，能够实现电机的闭环控制。同时在实际停车的时候，我们可以明显发现车辆会有一个向后略微倒车的趋势，这是因为闭环控制下，为了使得车辆从前进状态转变为静止状态，实际的闭环控制会导致控制电路中产生一个反向电动势，这一现象在此处无法展现，但确说明了我们的闭环控制方案是有效的。

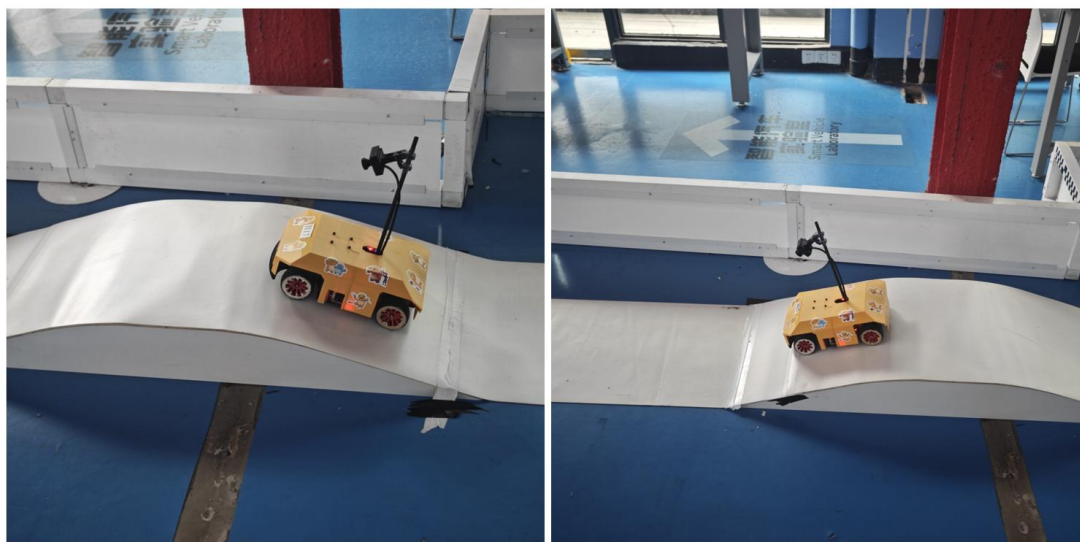


图 3.6.3 车辆放置于坡道上方效果